

Part01 :

 Project: 智慧邊緣監控下混合資料流之特定領域架構 (DSA) 設計與跨平台綜合 Trade-off 分析

1. Observation (觀察與痛點)

Observation(s):

在固定背景的智慧邊緣監控中，真實世界充滿了風吹、雨雪、光影等環境雜訊。前人（教授的論文）已證實，採用「傳統影像處理（Frame Difference + ATBS 自適應閾值降噪）」作為前端濾波器，再搭配後端「深度學習模型（CNN）」，能有效提升物件偵測的綜合準確度。

Issue（核心硬體痛點）：

然而，這種「傳統降噪邏輯 + 深度學習矩陣」的混合資料流（Hybrid Datapath）在傳統邊緣硬體部署時，會撞上嚴重的架構瓶頸。純 CNN 加速晶片（如 Hailo 8）無法處理非神經網路算子，導致資料必須在 Host CPU、片外記憶體（DDR）與 NPU 之間頻繁搬移更換上下文（Context Switch），產生嚴重的系統級延遲（Latency Wall）或記憶體牆（Memory Wall）帶來的功耗懲罰。

2. Objective (研究目標)

Objective(s):

設計一套專為此混合資料流量身打造的 FPGA SoC 特定領域架構（DSA）。透過 PS/PL 軟硬體任務切分與一條龍的客製化硬體流水線（Custom Pipeline），消除不必要的片外記憶體（DDR）存取，挑戰邊緣端運算效率的極限。

Model（對稱性模型選擇）：

精選兩款在工業界最具代表性、且在硬體本質上互為極端對照組的模型來測試本架構的適應性：

ResNet50：代表 High Arithmetic Intensity（高運算強度 / 計算密集型），用於模擬高精度、不容許漏報的關鍵安全場景（如高鐵落石）。

MobileNet：代表 Low Arithmetic Intensity（低運算強度 / 頻寬密集型），用於模擬大規模部署、電池供電的低功耗場景（如野生動物相機）。

3. Challenge（面臨的硬體挑戰）

PS/PL 任務切分與總線頻寬瓶頸：

如何在有限的 AXI 總線頻寬下，讓 PL 端的矩陣加速器與 PS 端的控制單元完美協調，且不讓資料傳輸成為新的瓶頸？

低運算強度模型的記憶體牆（Memory Wall）：

MobileNet 計算密度極低，晶片在經歷多個時脈週期（Timing Cycles）時，運算單元極易因缺乏資料而空轉（Pipeline Stalls），且頻繁存取外部 DDR 會導致功耗飆高。如何在 FPGA 內部設計客製化緩衝區（On-chip Buffer）來打破這道牆？

4. Method（研究方法）

Main idea & Insight:

學術洞察：晶片系統中最耗電與延遲的行為往往不是「計算」，而是「資料搬移（Data Movement）」。

核心想法：只要在 FPGA PL 端將「前端 ATBS 降噪 → 後端 MAC Array 矩陣計算」串接成一條龍的硬體流水線，就能達成「零拷貝（Zero-copy）」傳輸，將混合資料流

的優勢發揮到極致。

Method(s)（實作組合）：

建立一個 3×2 的跨平台實證矩陣進行橫向對比：

3 個硬體平台：FPGA SoC (你的 DSA) vs. Nvidia Jetson (通用邊緣 GPU) vs. Hailo 8 (專用 ASIC NPU)。

2 個神經網路：ResNet50 vs. MobileNet。

Justify（正當性與區隔）：

教授的研究 [5] 是巨觀的「純演算法與通用平台大排查」，而本研究是微觀的「晶片架構實作與底層行為分析」。此 3×2 框架非但不是盲目組合，更是為了從電腦架構（Computer Architecture）的角度，探討不同硬體體系結構在面對 Compute-bound 與 Memory-bound 兩個硬體極端時的架構適應性（Architectural Adaptability）。



5. Result（預期結果與分析方法）

Evaluate（綜合 Trade-off 評估指標）：

實時性與延遲：吞吐量（FPS）與首幀反應延遲（Time-to-First-Inference, ms）。

能效與功耗：整機功耗（System Power, Watt）與性能功耗比（FPS/Watt）。

精準度損失：降噪後的物件偵測精準度（mAP）與 INT8/FP16 量化損失。

資源與彈性：FPGA 資源佔用率（LUT/DSP/BRAM）與面對演算法變更時的硬體重構彈性。

Analyze（雙子星劇本分析）：

ResNet50 賽場：預期通用平台或 ASIC 雖然純算力強，但因頻繁與 Host CPU 換手而撞上 Latency Wall。本設計有望靠著客製化 Pipeline 消除傳輸延遲，在「高精度 +

「極致低延遲」的 Trade-off 中勝出。

MobileNet 賽場：預期通用平台因低運算強度而撞上 Memory Wall，核心空轉且 DDR 瘋狂耗電。本設計有望靠著客製化片上緩衝區減少外存，在「維持實時+極致低功耗（高能效比）」的 Trade-off 中勝出。

Conclusion（結論與決策指南）：

本研究最終不強求挑選單一「總冠軍」，而是提供一套工業界在面對不同場景限制（關鍵安全 vs. 大規模低功耗）時的邊緣端硬體佈署決策指南（Deployment Guideline

Part02 :

Project: An Energy-Efficient Hardware-Software Co-Designed DSA for Hybrid Vision-CNN Pipelines on FPGA SoC

└── Observation

| └── Observation(s): Hybrid datapath (FDM+ATBS + CNN) is highly accurate for edge object detection.

| └── Issue: Pure NPUs (like Hailo-8) cannot run FDM. Relying on Host CPU causes heavy off-chip DDR data transfers, leading to severe latency and high power (Memory Wall).

|── Objective

| └── Objective(s): Design an FPGA Domain-Specific Architecture (DSA) with a custom hardware pipeline to achieve "Zero-Copy" execution, eliminating off-chip memory bottlenecks.

| └── Model: ResNet50 (Compute-bound) vs. MobileNet (Memory-bound) as two extreme hardware stress tests.

|── Challenge: Designing custom on-chip BRAM buffers and PS/PL partitioning to prevent AXI bus bottlenecks.

|── Method

| └── Main idea & Insight: "Data movement costs more power than computation." Fusing FDM and CNN MAC into a single PL pipeline saves immense energy.

| └── Method(s): 3 Hardware platforms (FPGA, Jetson, Hailo) × 2 Models (ResNet, MobileNet).

| └── Justify: Elevates the work from "algorithm implementation" to "computer architecture design space exploration."

|── Result

| └── Evaluate: FPS/Watt (Energy Efficiency), Time-to-First-Inference (Latency), and Hardware Resources.

| └── Analyze: Prove FPGA beats ASIC/GPU in System-Level Latency (for ResNet) and FPS/Watt (for MobileNet).

| └── Conclusion: Deliver a hardware deployment guide for edge IoT based on power/latency constraints.

└── Reference

—— List: Roofline Model, PS/PL co-design, Hardware bottleneck analysis.

—— Paper: Your previous work on FDM+MobileNet (Serving as the baseline).

Part03 :

這兩者的底層物理機制與瓶頸成因梳理如下：

一、ResNet50 面臨的瓶頸：System-Level Latency Wall（系統級延遲牆）

1. 為什麼是 Latency 問題？

- 。 **計算密集特性（Compute-Bound）**：ResNet50 擁有大量標準的 3×3 與 1×1 卷積，運算強度（Arithmetic Intensity）高，資料重用率極佳。純論硬體乘加運算，GPU（Jetson）或專用 ASIC（Hailo-8）的純推論速度其實極快。
- 。 **致命痛點（混合資料流的跨晶片換手）**：
 - 。 專用 ASIC（如 Hailo-8）是純粹的神經網路引擎，無法處理 FDM（影格相減）與 ATBS（自適應閾值判斷）這類傳統視覺邏輯。
 - 。 整個資料流被迫拆解為：
 1. 視訊訊號進來，由 **Host CPU** 運算 FDM 與 ATBS 前處理。
 2. CPU 將過濾與裁切後的結果寫入主機記憶體（DDR）。
 3. 透過 PCIe 匯流排將資料跨晶片傳輸至 ASIC / 加速卡。
 4. ASIC 完成卷積推論，再將結果回傳。
 - 。 這段 **CPU 介入 $\rightarrow$ 驅動排程 $\rightarrow$ PCIe 跨晶片搬移** 所產生的「軟體與匯流排通訊開銷」，在端到端（End-to-End）時間中佔據了巨大比例。即使 ASIC 晶片只花 10 ms 算卷積，前面換手與搬移可能就耗損了數十毫秒。

2. 你的 FPGA DSA 如何在此擊敗對手？

- 。 **硬體流水線一條龍（Zero-Copy Pipeline）**：
在 FPGA 的 PL 端，影像串流進入後，硬體電路直接進行 FDM+ATBS 降噪，辨識出的感興趣區域（ROI）立即透過片上 FIFO / BRAM 直通 MAC 陣列，不依賴 Host

CPU、不走 PCIe、不落地到外部記憶體。

。 **勝出結果：** 雖然 FPGA 的純乘加時脈不如 GPU/ASIC，但消除了系統層級的換手等待與通訊延遲，在 **端到端首幀反應時間 (System-Level Latency)** 上成功勝出。

二、 MobileNet 面臨的瓶頸： Memory Wall 導致能效暴跌 (FPS/Watt 崩潰)

1. 為什麼 Depthwise Separable 會撞到 Memory Wall?

- 。 **運算強度極低 (Low Arithmetic Intensity / Memory-Bound)：**
 - 。 標準卷積是「一次搬移大量資料進晶片，在內部重複運算很久」。
 - 。 深度可分離卷積 (Depthwise Separable Convolution) 將卷積拆為 Depthwise (逐通道) 與 Pointwise (1×1)，大幅削減了參數量與計算量。
 - 。 **代價是計算密度極低：** 晶片每從外部 DDR 搬運 1 個 Byte 的特徵圖或權重，在晶片內部只執行極少次的乘加計算。
- 。 **傳統硬體 (GPU / ASIC) 的硬傷：**
 - 。 通用 GPU (Jetson) 或固定快取結構的 ASIC，其片上快取無法完全吻合 MobileNet 特殊的逐層特徵圖重用邏輯。
 - 。 為了計算下一層，硬體必須**頻繁存取外部 DDR 記憶體**。
 - 。 **功耗懲罰：** 在 CMOS 電路中，存取一次片外 DDR 的能量消耗是存取片上靜態記憶體 (SRAM/BRAM) 的百倍以上。同時，計算單元因為等不到 DDR 餵資料而頻繁處於「停頓空轉 (Pipeline Stall)」狀態，時脈樹 (Clock Tree) 卻持續翻轉耗電。這導致系統消耗了大量瓦特數，卻沒有產出對應的運算量，**FPS/Watt (能效比)** 嚴重惡化。

2. 你的 FPGA DSA 如何在此擊敗對手?

- 。 **客製化片上資料重用緩衝區 (Custom On-Chip Buffering)：**
 - 。 針對 MobileNet 的 Depthwise 特性，在 PL 端專門刻出尺寸精確匹配的 **Line Buffer** 與 **Ping-Pong Buffer** (使用 BRAM 實現)。
 - 。 讓前一層算完的中間特徵圖直接鎖在晶片內部的 BRAM 循環利用，極小化外部 DDR 的存取次數。
- 。 **勝出結果：** 外部 DDR 存取量大幅下降，直接斬斷了最主要的動態功耗來源；加上專屬 Pipeline 避免計算核心挨餓，整機功耗能壓制在極低水準，從而在 **FPS/Watt (性能功耗比)** 上取得顯著優勢。

■ 核心邏輯對照總結

測試模型	模型底層物理特徵	競爭對手 (GPU / ASIC) 撞上的硬體牆	你的 FPGA DSA 解決手段	最終勝利指標

ResNet50	計算密集 (High Arithmetic Intensity)	Latency Wall (延遲牆) ASIC 無法跑非神經網路前處理，被迫依賴 Host CPU 與 PCIe 來回換手搬移資料。	PL 端一條龍直通 FDM+ATBS 降噪與卷積在晶片內部串流直通，消滅 CPU 換手與通訊開銷。	System-Level Latency (端到端即時反應速度)
MobileNet	頻寬密集 (Low Arithmetic Intensity, DW 卷積頻繁存取)	Memory Wall (記憶體牆) 外部 DDR 頻繁讀寫消耗巨量傳輸功耗，且核心因缺資料停頓空轉。	客製化 BRAM 片上快取 專屬 Line Buffer 提高特徵圖重用率，斬斷外部 DDR 頻繁存取。	FPS/Watt (每瓦能效比)

Part04 :

在電腦架構與深度學習硬體加速中，「Compute-bound（受限於運算）」與「Memory-bound（受限於記憶體）」用來描述演算法在執行時，系統效能的瓶頸到底卡在『算力不夠』還是『頻寬不足』。

這兩個概念最經典的理論模型就是加州大學柏克萊分校提出的 **Roofline Model**（屋頂模型）。

一、核心裁判標準：運算強度 (Arithmetic Intensity)

決定一個任務會落在 Compute-bound 還是 Memory-bound，關鍵在於**運算強度 (Arithmetic Intensity, \$I\$)**：

$$I = \frac{\text{總運算量 (FLOPs)}}{\text{總記憶體存取量 (Bytes)}}$$

。 **白話解釋：**「硬體每從記憶體（DDR）搬 1 個 Byte 的資料進晶片，這個資料在晶片內部可以被拿來做幾次乘加計算（FLOPs）？」

每個硬體平台本身都有一個固定的「物理分水嶺 (Machine Balance, I_{crit})」：

$$I_{crit} = \frac{\text{硬體峰值算力 (Peak FLOPS)}}{\text{記憶體峰值頻寬 (Peak Memory Bandwidth)}}$$

- 當模型的 $I > I_{crit}$ $\rightarrow$ **Compute-bound**
- 當模型的 $I < I_{crit}$ $\rightarrow$ **Memory-bound**

二、Compute-bound (受限於計算)

- **定義：** 系統效能的上限取決於晶片內部**運算單元 (MAC Array / ALUs)** 的最大乘加速度。
- **物理行為：**
 - 資料搬進晶片內部後，會被極高度重複利用 (Data Reuse)。
 - 記憶體頻寬非常寬裕，能輕易把資料餵給運算核心。
 - 計算單元處於 100% 滿載運轉的狀態，想要提升速度，唯一的辦法是**增加更多運算核心 (MAC/DSP) 或拉高時脈**。
- **代表模型：ResNet50 (標準卷積)**
 - 標準卷積 (如 3×3 卷積) 中，同一個輸入像素與同一個權重，會與周圍相鄰的多個像素和通道反覆做卷積計算。
 - 它的運算強度極高 (可能每搬 1 Byte 資料進來，可以在晶片內做 20~50 次計算)。
 - 這類模型不會卡在記憶體速度，而是卡在「算術單元什麼時候能算完」。

三、Memory-bound (受限於記憶體頻寬)

- **定義：** 系統效能的上限取決於**外部記憶體匯流排 (DDR Bus)** 的傳輸頻寬。
- **物理行為：**
 - 資料的重複利用率極低。
 - 計算單元算得太快，但外部記憶體搬運速度太慢，導致運算核心大部分時間都在**停頓空轉 (Pipeline Stall)**，等記憶體把下一筆資料餵進來。
 - 此時就算你給它再多、再強的 MAC 單元，效能也完全不會提升，因為瓶頸卡在「資料進不來」。
- **代表模型：MobileNet (深度可分離卷積 Depthwise Separable Conv)**
 - MobileNet 為了輕量化，採用了 Depthwise (DW) 卷積。DW 卷積是每個輸入通道只配一個 3×3 的卷積核，通道之間彼此不互動。
 - 計算量 (FLOPs) 雖然大幅縮減，但每做一次計算就需要去外部記憶體讀取特徵圖與權重，**資料根本沒在晶片內重複利用就被丟棄了**。
 - 它的運算強度極低 (可能每搬 1 Byte 資料，內部只算不到 2 次乘加)。
 - 這導致它在硬體上徹底撞上 **Memory Wall (記憶體牆)**：運算單元頻繁挨餓，記憶體匯流排卻瘋狂滿載搬資料，導致「效能低、功耗大 (因為頻繁存取 DDR 極其耗電)」。

■ 用生活化例子對比

概念	料理比喻	硬體狀態	你的論文對應點
Compute-bound (以 ResNet 為代表)	廚房助手（記憶體）送來一塊大肉，大廚（運算單元）需要花 30 分鐘精雕細琢雕刻（大量運算）。助手送菜非常輕鬆，大廚的手一刻沒停過。	計算核心滿載，記憶體頻寬不是瓶頸。	傳統 GPU/ASIC 算得很快，但因為缺乏前端降噪管線，撞上 CPU 換手的 Latency Wall。
Memory-bound (以 MobileNet 為代表)	廚房助手需要跑到地下室倉庫搬一整箱洋蔥，大廚切兩刀就切完了，然後坐在砧板前乾等助手搬下一箱。大廚大部分時間都在發呆，助手累得半死。	計算核心頻繁空轉等待，外部 DDR 頻繁搬移資料消耗巨量功耗。	傳統架構撞上 Memory Wall，你的 FPGA DSA 透過在 PL 端刻專屬的 Line Buffer 解決這個問題。